

Security & Governance

Comprehensive Guide to Securing Data Ingestion Pipelines

Table of Contents

1. [Encryption](#)
 2. [Authentication & Authorization](#)
 3. [Data Lineage](#)
 4. [Data Quality](#)
 5. [Compliance Frameworks](#)
 6. [Audit Logging](#)
 7. [Data Masking & PII Protection](#)
 8. [Security Best Practices](#)
-

1. Encryption {#encryption}

Encryption at Rest

Kafka Encryption at Rest:

```
# server.properties (broker configuration)

# Enable encryption for log segments
log.dirs=/var/lib/kafka/data-encrypted

# Encryption settings (filesystem-level)
# Use encrypted volumes (LUKS, dm-crypt, AWS EBS encryption, etc.)
```

Application-level Encryption:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import java.security.SecureRandom;
import java.util.Base64;

public class DataEncryptor {

    private static final String ALGORITHM = "AES/GCM/NoPadding";
    private static final int GCM_TAG_LENGTH = 128;
    private static final int GCM_IV_LENGTH = 12;

    private final SecretKey secretKey;
```

```
public DataEncryptor(SecretKey key) {
    this.secretKey = key;
}

/**
 * Encrypt sensitive data before sending to Kafka
 */
public String encrypt(String plaintext) throws Exception {
    // Generate random IV
    byte[] iv = new byte[GCM_IV_LENGTH];
    SecureRandom random = new SecureRandom();
    random.nextBytes(iv);

    // Initialize cipher
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    GCMParameterSpec parameterSpec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, parameterSpec);

    // Encrypt
    byte[] ciphertext = cipher.doFinal(plaintext.getBytes("UTF-8"));

    // Combine IV + ciphertext
    byte[] combined = new byte[iv.length + ciphertext.length];
    System.arraycopy(iv, 0, combined, 0, iv.length);
    System.arraycopy(ciphertext, 0, combined, iv.length, ciphertext.length);

    // Base64 encode
    return Base64.getEncoder().encodeToString(combined);
}

/**
 * Decrypt data after reading from Kafka
 */
public String decrypt(String encrypted) throws Exception {
    // Base64 decode
    byte[] combined = Base64.getDecoder().decode(encrypted);

    // Extract IV and ciphertext
    byte[] iv = new byte[GCM_IV_LENGTH];
    byte[] ciphertext = new byte[combined.length - GCM_IV_LENGTH];
    System.arraycopy(combined, 0, iv, 0, iv.length);
    System.arraycopy(combined, iv.length, ciphertext, 0, ciphertext.length);

    // Initialize cipher
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    GCMParameterSpec parameterSpec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);
    cipher.init(Cipher.DECRYPT_MODE, secretKey, parameterSpec);

    // Decrypt
    byte[] plaintext = cipher.doFinal(ciphertext);

    return new String(plaintext, "UTF-8");
}
```

```
/**
 * Generate encryption key
 */
public static SecretKey generateKey() throws Exception {
    KeyGenerator keyGenerator = KeyGenerator.getInstance("AES");
    keyGenerator.init(256);
    return keyGenerator.generateKey();
}

public static void main(String[] args) throws Exception {
    // Generate key (store securely in KMS)
    SecretKey key = generateKey();

    DataEncryptor encryptor = new DataEncryptor(key);

    // Encrypt sensitive data
    String sensitive = "SSN: 123-45-6789, Credit Card: 4111111111111111";
    String encrypted = encryptor.encrypt(sensitive);
    System.out.println("Encrypted: " + encrypted);

    // Decrypt
    String decrypted = encryptor.decrypt(encrypted);
    System.out.println("Decrypted: " + decrypted);
}
}
```

Encryption in Transit

Kafka SSL/TLS Configuration:

Broker Configuration (`server.properties`):

```
# Enable SSL
listeners=SSL://kafka-broker:9093
advertised.listeners=SSL://kafka-broker.example.com:9093

# SSL settings
ssl.keystore.location=/var/private/ssl/kafka.server.keystore.jks
ssl.keystore.password=keystore-password
ssl.key.password=key-password
ssl.truststore.location=/var/private/ssl/kafka.server.truststore.jks
ssl.truststore.password=truststore-password

# Client authentication
ssl.client.auth=required

# SSL protocol
ssl.enabled.protocols=TLSv1.2,TLSv1.3
ssl.protocol=TLSv1.3
```

```
# Cipher suites
ssl.cipher.suites=TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
```

Producer Configuration:

```
Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka-
broker.example.com:9093");

// SSL settings
props.put("security.protocol", "SSL");
props.put("ssl.truststore.location", "/path/to/client.truststore.jks");
props.put("ssl.truststore.password", "truststore-password");
props.put("ssl.keystore.location", "/path/to/client.keystore.jks");
props.put("ssl.keystore.password", "keystore-password");
props.put("ssl.key.password", "key-password");

KafkaProducer<String, String> producer = new KafkaProducer<>(props);
```

Generate SSL Certificates:

```
#!/bin/bash

# Variables
VALIDITY_DAYS=365
KEYSTORE_PASSWORD="changeit"
TRUSTSTORE_PASSWORD="changeit"
KEY_PASSWORD="changeit"

# 1. Generate CA (Certificate Authority)
openssl req -new -x509 -keyout ca-key -out ca-cert -days $VALIDITY_DAYS \
  -subj "/CN=KafkaCA" \
  -passout pass:$KEY_PASSWORD

# 2. Create truststore and import CA cert
keytool -keystore kafka.server.truststore.jks -alias CARoot -import \
  -file ca-cert \
  -storepass $TRUSTSTORE_PASSWORD \
  -noprompt

# 3. Create keystore for broker
keytool -keystore kafka.server.keystore.jks -alias localhost -validity $VALIDITY_DAYS \
  -genkey -keyalg RSA \
  -dname "CN=kafka-
broker.example.com,OU=Engineering,O=Company,L=City,ST=State,C=US" \
  -storepass $KEYSTORE_PASSWORD \
  -keypass $KEY_PASSWORD
```

```
# 4. Create certificate signing request (CSR)
keytool -keystore kafka.server.keystore.jks -alias localhost \
  -certreq -file cert-file \
  -storepass $KEYSTORE_PASSWORD

# 5. Sign the certificate with CA
openssl x509 -req -CA ca-cert -CAkey ca-key -in cert-file \
  -out cert-signed -days $VALIDITY_DAYS -CAcreateserial \
  -passin pass:$KEY_PASSWORD

# 6. Import CA cert into keystore
keytool -keystore kafka.server.keystore.jks -alias CARoot \
  -import -file ca-cert \
  -storepass $KEYSTORE_PASSWORD \
  -noprompt

# 7. Import signed certificate into keystore
keytool -keystore kafka.server.keystore.jks -alias localhost \
  -import -file cert-signed \
  -storepass $KEYSTORE_PASSWORD

echo "SSL certificates generated successfully!"
```

2. Authentication & Authorization {#auth}

Kafka SASL/SCRAM Authentication

Broker Configuration:

```
# server.properties

# Listeners with SASL
listeners=SASL_SSL://kafka-broker:9093
advertised.listeners=SASL_SSL://kafka-broker.example.com:9093

# SASL mechanism
sasl.enabled.mechanisms=SCRAM-SHA-512
sasl.mechanism.inter.broker.protocol=SCRAM-SHA-512

# JAAS configuration
listener.name.sasl_ssl.scram-sha-512.sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule
required \
  username="admin" \
  password="admin-secret";

# Authorization
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
super.users=User:admin
```

Create Users:

```
# Create user with SCRAM-SHA-512
kafka-configs.sh --bootstrap-server localhost:9092 \
  --alter \
  --add-config 'SCRAM-SHA-512=[password=user1-password]' \
  --entity-type users \
  --entity-name user1
```

Producer/Consumer Configuration:

```
Properties props = new Properties();
props.put("bootstrap.servers", "kafka-broker.example.com:9093");
props.put("security.protocol", "SASL_SSL");
props.put("sasl.mechanism", "SCRAM-SHA-512");
props.put("sasl.jaas.config",
  "org.apache.kafka.common.security.scram.ScramLoginModule required " +
  "username=\"user1\" " +
  "password=\"user1-password\";");

// SSL settings
props.put("ssl.truststore.location", "/path/to/truststore.jks");
props.put("ssl.truststore.password", "truststore-password");
```

Kafka ACLs (Access Control Lists)

Grant Permissions:

```
# Allow user1 to READ from topic 'events'
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:user1 \
  --operation Read \
  --topic events \
  --group consumer-group-1

# Allow user2 to WRITE to topic 'events'
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:user2 \
  --operation Write \
  --topic events

# Allow user3 to CREATE topics
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:user3 \
```

```

--operation Create \
--cluster

# List ACLs
kafka-acls.sh --bootstrap-server localhost:9092 \
--list \
--topic events

# Remove ACL
kafka-acls.sh --bootstrap-server localhost:9092 \
--remove \
--allow-principal User:user1 \
--operation Read \
--topic events

```

Role-Based Access Control (RBAC)

File: `rbac_manager.py`

```

from enum import Enum
from typing import Set, Dict
import json

class Permission(Enum):
    """Data pipeline permissions"""
    READ = "read"
    WRITE = "write"
    ADMIN = "admin"
    DELETE = "delete"

class Role(Enum):
    """Predefined roles"""
    DATA_ENGINEER = "data_engineer"
    DATA_ANALYST = "data_analyst"
    DATA_SCIENTIST = "data_scientist"
    ADMIN = "admin"

class RBACManager:
    """
    Role-Based Access Control for data pipelines
    """

    def __init__(self):
        # Role-to-permissions mapping
        self.role_permissions: Dict[Role, Set[Permission]] = {
            Role.DATA_ANALYST: {Permission.READ},
            Role.DATA_SCIENTIST: {Permission.READ, Permission.WRITE},
            Role.DATA_ENGINEER: {Permission.READ, Permission.WRITE,
            Permission.DELETE},
            Role.ADMIN: {Permission.READ, Permission.WRITE, Permission.DELETE,
            Permission.ADMIN}

```

```

    }

    # User-to-roles mapping
    self.user_roles: Dict[str, Set[Role]] = {}

    # Resource-level permissions
    self.resource_permissions: Dict[str, Dict[str, Set[Permission]]] = {}

    def assign_role(self, user: str, role: Role):
        """Assign role to user"""
        if user not in self.user_roles:
            self.user_roles[user] = set()
        self.user_roles[user].add(role)

    def revoke_role(self, user: str, role: Role):
        """Revoke role from user"""
        if user in self.user_roles:
            self.user_roles[user].discard(role)

    def has_permission(self, user: str, permission: Permission, resource: str =
None) -> bool:
        """
        Check if user has permission

        Args:
            user: Username
            permission: Required permission
            resource: Optional specific resource (e.g., topic name)

        Returns:
            True if user has permission
        """
        # Check resource-specific permissions first
        if resource and resource in self.resource_permissions:
            if user in self.resource_permissions[resource]:
                if permission in self.resource_permissions[resource][user]:
                    return True

        # Check role-based permissions
        if user not in self.user_roles:
            return False

        user_permissions = set()
        for role in self.user_roles[user]:
            user_permissions.update(self.role_permissions.get(role, set()))

        return permission in user_permissions

    def grant_resource_permission(self, user: str, resource: str, permission:
Permission):
        """Grant permission on specific resource"""
        if resource not in self.resource_permissions:
            self.resource_permissions[resource] = {}
        if user not in self.resource_permissions[resource]:

```

```

        self.resource_permissions[resource][user] = set()

        self.resource_permissions[resource][user].add(permission)

def check_access(self, user: str, action: str, resource: str) -> bool:
    """
    Check if user can perform action on resource

    Example:
        check_access("alice", "read", "topic:user-events")
    """
    permission_map = {
        "read": Permission.READ,
        "write": Permission.WRITE,
        "delete": Permission.DELETE,
        "admin": Permission.ADMIN
    }

    required_permission = permission_map.get(action.lower())
    if not required_permission:
        return False

    return self.has_permission(user, required_permission, resource)

def audit_log(self, user: str, action: str, resource: str, allowed: bool):
    """Log access attempts"""
    import logging
    logger = logging.getLogger(__name__)

    status = "ALLOWED" if allowed else "DENIED"
    logger.info(f"ACCESS {status}: user={user}, action={action}, resource={resource}")

# Example usage
if __name__ == "__main__":
    rbac = RBACManager()

    # Assign roles
    rbac.assign_role("alice", Role.DATA_ENGINEER)
    rbac.assign_role("bob", Role.DATA_ANALYST)
    rbac.assign_role("charlie", Role.ADMIN)

    # Grant specific resource permissions
    rbac.grant_resource_permission("bob", "topic:public-events", Permission.WRITE)

    # Check permissions
    print(f"Alice can write: {rbac.has_permission('alice', Permission.WRITE)}") # True
    print(f"Bob can write: {rbac.has_permission('bob', Permission.WRITE)}") # False
    print(f"Bob can write to public-events: {rbac.has_permission('bob', Permission.WRITE, 'topic:public-events')}") # True

    # Check access with audit

```

```
allowed = rbac.check_access("alice", "write", "topic:user-events")
rbac.audit_log("alice", "write", "topic:user-events", allowed)
```

3. Data Lineage {#lineage}

Lineage Tracking

File: `lineage_tracker.py`

```
from dataclasses import dataclass, field
from typing import List, Dict, Optional
from datetime import datetime
import json

@dataclass
class DataAsset:
    """Represents a data asset (table, topic, file, etc.)"""
    id: str
    name: str
    type: str # 'topic', 'table', 'file', 'api'
    location: str
    schema: Optional[Dict] = None
    metadata: Dict = field(default_factory=dict)

@dataclass
class Transformation:
    """Represents a data transformation"""
    id: str
    name: str
    type: str # 'filter', 'join', 'aggregate', 'map'
    code: Optional[str] = None
    parameters: Dict = field(default_factory=dict)

@dataclass
class LineageNode:
    """Node in lineage graph"""
    asset: DataAsset
    upstream: List['LineageNode'] = field(default_factory=list)
    transformation: Optional[Transformation] = None
    timestamp: datetime = field(default_factory=datetime.now)

class LineageTracker:
    """
    Track data lineage across pipeline
    """

    def __init__(self):
        self.lineage_graph: Dict[str, LineageNode] = {}

    def register_asset(self, asset: DataAsset) -> LineageNode:
```

```

    """Register a data asset"""
    if asset.id not in self.lineage_graph:
        node = LineageNode(asset=asset)
        self.lineage_graph[asset.id] = node
    return self.lineage_graph[asset.id]

def record_transformation(
    self,
    source_assets: List[DataAsset],
    target_asset: DataAsset,
    transformation: Transformation
):
    """
    Record a transformation from source(s) to target

    Args:
        source_assets: Input datasets
        target_asset: Output dataset
        transformation: Transformation applied
    """
    # Register target
    target_node = self.register_asset(target_asset)
    target_node.transformation = transformation

    # Link upstream sources
    for source in source_assets:
        source_node = self.register_asset(source)
        target_node.upstream.append(source_node)

def get_upstream_lineage(self, asset_id: str, max_depth: int = 10) ->
List[LineageNode]:
    """Get all upstream dependencies"""
    if asset_id not in self.lineage_graph:
        return []

    result = []
    visited = set()

    def traverse(node: LineageNode, depth: int):
        if depth > max_depth or node.asset.id in visited:
            return

        visited.add(node.asset.id)
        result.append(node)

        for upstream_node in node.upstream:
            traverse(upstream_node, depth + 1)

    traverse(self.lineage_graph[asset_id], 0)
    return result

def get_downstream_lineage(self, asset_id: str, max_depth: int = 10) ->
List[LineageNode]:
    """Get all downstream consumers"""

```

```

        if asset_id not in self.lineage_graph:
            return []

        result = []
        visited = set()

        def traverse(target_id: str, depth: int):
            if depth > max_depth or target_id in visited:
                return

            visited.add(target_id)

            # Find nodes that have this asset as upstream
            for node_id, node in self.lineage_graph.items():
                for upstream_node in node.upstream:
                    if upstream_node.asset.id == target_id:
                        result.append(node)
                        traverse(node.asset.id, depth + 1)

        traverse(asset_id, 0)
        return result

    def export_lineage(self, asset_id: str) -> Dict:
        """Export lineage as JSON"""
        upstream = self.get_upstream_lineage(asset_id)
        downstream = self.get_downstream_lineage(asset_id)

        return {
            'asset_id': asset_id,
            'upstream': [self._node_to_dict(n) for n in upstream],
            'downstream': [self._node_to_dict(n) for n in downstream],
            'generated_at': datetime.now().isoformat()
        }

    def _node_to_dict(self, node: LineageNode) -> Dict:
        """Convert node to dictionary"""
        return {
            'asset': {
                'id': node.asset.id,
                'name': node.asset.name,
                'type': node.asset.type,
                'location': node.asset.location
            },
            'transformation': {
                'name': node.transformation.name,
                'type': node.transformation.type
            } if node.transformation else None,
            'timestamp': node.timestamp.isoformat()
        }

# Example usage
if __name__ == "__main__":
    tracker = LineageTracker()

```

```

# Source: Kafka topic
source_topic = DataAsset(
    id='kafka://user-events',
    name='user-events',
    type='topic',
    location='kafka:9092'
)

# Intermediate: Processed data
processed = DataAsset(
    id='s3://processed/events',
    name='processed-events',
    type='file',
    location='s3://processed/events/'
)

# Target: Data warehouse table
warehouse = DataAsset(
    id='postgres://warehouse/events',
    name='events_fact',
    type='table',
    location='postgres://warehouse:5432/analytics'
)

# Record transformations
tracker.record_transformation(
    source_assets=[source_topic],
    target_asset=processed,
    transformation=Transformation(
        id='transform-1',
        name='Filter and Enrich',
        type='map',
        code='events.filter(amount > 0).enrich(user_info)'
    )
)

tracker.record_transformation(
    source_assets=[processed],
    target_asset=warehouse,
    transformation=Transformation(
        id='transform-2',
        name='Load to Warehouse',
        type='load',
        parameters={'batch_size': 10000}
    )
)

# Query lineage
lineage = tracker.export_lineage('postgres://warehouse/events')
print(json.dumps(lineage, indent=2))

```

File: atlas_integration.py

```
from atlasclient.client import Atlas
from atlasclient.models import Entity, EntityTypeDef
import json

class AtlasLineageIntegration:
    """
    Integrate with Apache Atlas for lineage tracking
    """

    def __init__(self, atlas_url: str, username: str, password: str):
        self.client = Atlas(atlas_url, username=username, password=password)

    def register_kafka_topic(self, topic_name: str, bootstrap_servers: str):
        """Register Kafka topic in Atlas"""
        entity = Entity({
            'typeName': 'kafka_topic',
            'attributes': {
                'qualifiedName': f'{topic_name}@{bootstrap_servers}',
                'name': topic_name,
                'clusterName': bootstrap_servers,
                'uri': f'kafka://{bootstrap_servers}/{topic_name}'
            }
        })

        return self.client.entity_post.create(data=entity)

    def register_table(self, database: str, table: str, columns: list):
        """Register database table in Atlas"""
        entity = Entity({
            'typeName': 'rdbms_table',
            'attributes': {
                'qualifiedName': f'{database}.{table}',
                'name': table,
                'db': {'qualifiedName': database},
                'columns': columns
            }
        })

        return self.client.entity_post.create(data=entity)

    def register_process(self, name: str, inputs: list, outputs: list):
        """Register ETL process with lineage"""
        entity = Entity({
            'typeName': 'Process',
            'attributes': {
                'qualifiedName': name,
                'name': name,
                'inputs': inputs,
                'outputs': outputs
            }
        })
    })
```

```

        return self.client.entity_post.create(data=entity)

# Example usage
if __name__ == "__main__":
    atlas = AtlasLineageIntegration(
        atlas_url='http://atlas:21000',
        username='admin',
        password='admin'
    )

    # Register Kafka topic
    topic = atlas.register_kafka_topic('user-events', 'kafka:9092')

    # Register warehouse table
    table = atlas.register_table(
        database='analytics',
        table='events_fact',
        columns=['user_id', 'event_type', 'timestamp', 'amount']
    )

    # Register process (lineage)
    process = atlas.register_process(
        name='event-ingestion-pipeline',
        inputs=[topic['guid']],
        outputs=[table['guid']]
    )

```

4. Data Quality {#data-quality}

Data Quality Framework

File: `data_quality.py`

```

from abc import ABC, abstractmethod
from typing import Any, Dict, List
from dataclasses import dataclass
import pandas as pd

@dataclass
class QualityCheckResult:
    """Result of a quality check"""
    check_name: str
    passed: bool
    message: str
    severity: str # 'error', 'warning', 'info'
    details: Dict = None

class QualityCheck(ABC):
    """Base class for quality checks"""

```

```

    @abstractmethod
    def check(self, data: pd.DataFrame) -> QualityCheckResult:
        """Execute quality check"""
        pass

class NotNullCheck(QualityCheck):
    """Check for null values in column"""

    def __init__(self, column: str, severity: str = 'error'):
        self.column = column
        self.severity = severity

    def check(self, data: pd.DataFrame) -> QualityCheckResult:
        null_count = data[self.column].isnull().sum()
        total = len(data)
        passed = null_count == 0

        return QualityCheckResult(
            check_name=f'NotNull_{self.column}',
            passed=passed,
            message=f'{null_count} null values found in {self.column} '
            f'({null_count/total*100:.2f}%)',
            severity=self.severity if not passed else 'info',
            details={'null_count': int(null_count), 'total': int(total)}
        )

class UniquenessCheck(QualityCheck):
    """Check for duplicate values"""

    def __init__(self, columns: List[str], severity: str = 'error'):
        self.columns = columns
        self.severity = severity

    def check(self, data: pd.DataFrame) -> QualityCheckResult:
        duplicate_count = data.duplicated(subset=self.columns).sum()
        total = len(data)
        passed = duplicate_count == 0

        return QualityCheckResult(
            check_name=f'Uniqueness_{"".join(self.columns)}',
            passed=passed,
            message=f'{duplicate_count} duplicates found '
            f'({duplicate_count/total*100:.2f}%)',
            severity=self.severity if not passed else 'info',
            details={'duplicate_count': int(duplicate_count), 'total': int(total)}
        )

class RangeCheck(QualityCheck):
    """Check if values are within valid range"""

    def __init__(self, column: str, min_value: float, max_value: float, severity:
str = 'warning'):
        self.column = column

```

```

        self.min_value = min_value
        self.max_value = max_value
        self.severity = severity

    def check(self, data: pd.DataFrame) -> QualityCheckResult:
        out_of_range = ((data[self.column] < self.min_value) | (data[self.column]
> self.max_value)).sum()
        total = len(data)
        passed = out_of_range == 0

        return QualityCheckResult(
            check_name=f'Range_{self.column}',
            passed=passed,
            message=f'{out_of_range} values out of range [{self.min_value},
{self.max_value}]',
            severity=self.severity if not passed else 'info',
            details={ 'out_of_range_count': int(out_of_range), 'total': int(total)}
        )

class SchemaCheck(QualityCheck):
    """Check if schema matches expected schema"""

    def __init__(self, expected_schema: Dict[str, str], severity: str = 'error'):
        self.expected_schema = expected_schema
        self.severity = severity

    def check(self, data: pd.DataFrame) -> QualityCheckResult:
        actual_schema = {col: str(dtype) for col, dtype in data.dtypes.items()}

        missing_columns = set(self.expected_schema.keys()) -
set(actual_schema.keys())
        extra_columns = set(actual_schema.keys()) -
set(self.expected_schema.keys())
        type_mismatches = []

        for col in set(actual_schema.keys()) & set(self.expected_schema.keys()):
            if actual_schema[col] != self.expected_schema[col]:
                type_mismatches.append(f'{col}: expected
{self.expected_schema[col]}, got {actual_schema[col]}')

        passed = len(missing_columns) == 0 and len(extra_columns) == 0 and
len(type_mismatches) == 0

        message_parts = []
        if missing_columns:
            message_parts.append(f'Missing columns: {missing_columns}')
        if extra_columns:
            message_parts.append(f'Extra columns: {extra_columns}')
        if type_mismatches:
            message_parts.append(f'Type mismatches: {type_mismatches}')

        return QualityCheckResult(
            check_name='Schema',
            passed=passed,

```

```

        message='; '.join(message_parts) if message_parts else 'Schema
matches',
        severity=self.severity if not passed else 'info',
        details={
            'missing_columns': list(missing_columns),
            'extra_columns': list(extra_columns),
            'type_mismatches': type_mismatches
        }
    )

class DataQualityValidator:
    """
    Run multiple quality checks on data
    """

    def __init__(self):
        self.checks: List[QualityCheck] = []

    def add_check(self, check: QualityCheck):
        """Add a quality check"""
        self.checks.append(check)

    def validate(self, data: pd.DataFrame) -> List[QualityCheckResult]:
        """Run all checks"""
        results = []
        for check in self.checks:
            result = check.check(data)
            results.append(result)
        return results

    def validate_and_raise(self, data: pd.DataFrame):
        """Validate and raise exception if any error-level checks fail"""
        results = self.validate(data)

        errors = [r for r in results if not r.passed and r.severity == 'error']

        if errors:
            error_messages = [f'{r.check_name}: {r.message}' for r in errors]
            raise ValueError(f'Data quality checks failed:\n' +
                             '\n'.join(error_messages))

        # Log warnings
        warnings = [r for r in results if not r.passed and r.severity ==
                    'warning']
        for warning in warnings:
            print(f'WARNING - {warning.check_name}: {warning.message}')

        return results

# Example usage
if __name__ == "__main__":
    # Sample data
    data = pd.DataFrame({
        'user_id': [1, 2, 3, 4, None],

```

```

        'amount': [100, 200, -50, 150, 175],
        'email': ['a@example.com', 'b@example.com', 'c@example.com',
                  'd@example.com', 'e@example.com']
    })

    # Create validator
    validator = DataQualityValidator()

    # Add checks
    validator.add_check(NotNullCheck('user_id'))
    validator.add_check(UniquenessCheck(['email']))
    validator.add_check(RangeCheck('amount', min_value=0, max_value=1000))
    validator.add_check(SchemaCheck({
        'user_id': 'float64',
        'amount': 'int64',
        'email': 'object'
    }))

    # Validate
    try:
        results = validator.validate_and_raise(data)
        print("Data quality validation passed!")
    except ValueError as e:
        print(f"Validation failed: {e}")

```

[Document continues with sections 5-8 covering Compliance, Audit Logging, Data Masking, and Best Practices...]

Document Status

Current Size: ~40KB **Completion:** Part 1 of 2

5. Compliance Frameworks {#compliance}

GDPR Compliance

Data Subject Rights Implementation:

```

class GDPRComplianceHandler:
    """
    Handle GDPR compliance requirements
    """

    def __init__(self, kafka_producer, database_conn):
        self.producer = kafka_producer
        self.db = database_conn

    def right_to_access(self, user_id: str) -> Dict:
        """
        Right to Access: Export all data for a user

```

```

Article 15 GDPR
"""
cursor = self.db.cursor()

# Collect all user data
user_data = {}

# Personal information
cursor.execute("SELECT * FROM users WHERE user_id = %s", (user_id,))
user_data['personal_info'] = cursor.fetchone()

# Transaction history
cursor.execute("SELECT * FROM orders WHERE user_id = %s", (user_id,))
user_data['orders'] = cursor.fetchall()

# Event logs
cursor.execute("SELECT * FROM events WHERE user_id = %s LIMIT 1000",
(user_id,))
user_data['events'] = cursor.fetchall()

# Generate export file
export = {
    'user_id': user_id,
    'exported_at': datetime.now().isoformat(),
    'data': user_data
}

return export

def right_to_erasure(self, user_id: str):
    """
    Right to Erasure (Right to be Forgotten)
    Article 17 GDPR
    """
    cursor = self.db.cursor()

    # Anonymize personal data (don't delete, for audit trail)
    cursor.execute("""
        UPDATE users
        SET name = 'DELETED',
            email = CONCAT('deleted_', user_id, '@deleted.com'),
            phone = NULL,
            address = NULL,
            deleted_at = NOW()
        WHERE user_id = %s
    """, (user_id,))

    # Send deletion event to Kafka
    deletion_event = {
        'event_type': 'user_deletion',
        'user_id': user_id,
        'timestamp': datetime.now().isoformat(),
        'reason': 'gdpr_right_to_erasure'
    }

```

```

        self.producer.send('user-deletions', deletion_event)

        self.db.commit()

        # Trigger downstream deletion in data lake, warehouses, etc.
        self._trigger_downstream_deletion(user_id)

def right_to_rectification(self, user_id: str, corrections: Dict):
    """
    Right to Rectification: Update incorrect data
    Article 16 GDPR
    """
    cursor = self.db.cursor()

    # Build UPDATE query
    set_clauses = []
    values = []

    for field, value in corrections.items():
        set_clauses.append(f"{field} = %s")
        values.append(value)

    values.append(user_id)

    query = f"""
        UPDATE users
        SET {' '.join(set_clauses)},
            updated_at = NOW()
        WHERE user_id = %s
    """

    cursor.execute(query, values)
    self.db.commit()

    # Log rectification for audit
    self._log_rectification(user_id, corrections)

def right_to_portability(self, user_id: str) -> bytes:
    """
    Right to Data Portability: Export in machine-readable format
    Article 20 GDPR
    """
    data = self.right_to_access(user_id)

    # Export as JSON (machine-readable)
    import json
    json_data = json.dumps(data, indent=2, default=str)

    return json_data.encode('utf-8')

def _trigger_downstream_deletion(self, user_id: str):
    """Trigger deletion in downstream systems"""
    # Send to deletion queue

```

```

        deletion_message = {
            'user_id': user_id,
            'timestamp': datetime.now().isoformat(),
            'systems': ['data_lake', 'warehouse', 'analytics', 'ml_models']
        }

        self.producer.send('deletion-queue', deletion_message)

    def _log_rectification(self, user_id: str, corrections: Dict):
        """Log rectification for audit trail"""
        cursor = self.db.cursor()
        cursor.execute("""
            INSERT INTO audit_log (user_id, action, details, timestamp)
            VALUES (%s, 'rectification', %s, NOW())
            """, (user_id, json.dumps(corrections)))
        self.db.commit()

```

CCPA Compliance

```

class CCPAComplianceHandler:
    """
    California Consumer Privacy Act compliance
    """

    def do_not_sell_request(self, user_id: str):
        """
        Handle "Do Not Sell My Information" request
        """
        cursor = self.db.cursor()

        # Update user preferences
        cursor.execute("""
            UPDATE user_preferences
            SET do_not_sell = TRUE,
                updated_at = NOW()
            WHERE user_id = %s
            """, (user_id,))

        self.db.commit()

        # Notify third-party data processors
        self._notify_do_not_sell(user_id)

    def opt_out_request(self, user_id: str, category: str):
        """
        Handle opt-out request for specific data category
        Categories: advertising, analytics, etc.
        """
        cursor = self.db.cursor()

        cursor.execute("""

```

```

        INSERT INTO opt_outs (user_id, category, opted_out_at)
        VALUES (%s, %s, NOW())
        ON CONFLICT (user_id, category)
        DO UPDATE SET opted_out_at = NOW()
        """ , (user_id, category))

    self.db.commit()

```

HIPAA Compliance (Healthcare)

```

class HIPAAComplianceHandler:
    """
    Health Insurance Portability and Accountability Act compliance
    """

    def __init__(self):
        self.phi_fields = [
            'patient_name', 'ssn', 'medical_record_number',
            'health_plan_id', 'address', 'phone', 'email',
            'biometric_identifiers', 'photos'
        ]

    def de_identify_phi(self, record: Dict) -> Dict:
        """
        De-identify Protected Health Information (PHI)
        Safe Harbor method (HIPAA Privacy Rule)
        """
        de_identified = record.copy()

        # Remove direct identifiers
        for field in self.phi_fields:
            if field in de_identified:
                de_identified[field] = None

        # Generalize dates (remove day)
        if 'date_of_birth' in de_identified:
            dob = de_identified['date_of_birth']
            if dob:
                de_identified['date_of_birth'] = dob.replace(day=1)

        # Generalize ages > 89
        if 'age' in de_identified and de_identified['age'] > 89:
            de_identified['age'] = '90+'

        # Generalize geography (remove details beyond state)
        if 'zip_code' in de_identified:
            zip_code = de_identified['zip_code']
            if zip_code:
                de_identified['zip_code'] = zip_code[:3] + '00' # First 3 digits

```

only

```

        return de_identified

    def audit_phi_access(self, user: str, patient_id: str, action: str):
        """
        Audit all PHI access (HIPAA requirement)
        """
        audit_record = {
            'timestamp': datetime.now().isoformat(),
            'user': user,
            'patient_id': patient_id,
            'action': action,
            'ip_address': self._get_client_ip(),
            'system': 'data_pipeline'
        }

        # Write to immutable audit log
        self._write_audit_log(audit_record)

```

6. Audit Logging {#audit-logging}

Comprehensive Audit Log System

File: `audit_logger.py`

```

import logging
import json
from datetime import datetime
from typing import Dict, Any
from confluent_kafka import Producer
import hashlib

class AuditLogger:
    """
    Comprehensive audit logging system
    """

    def __init__(self, kafka_bootstrap_servers: str):
        self.producer = Producer({
            'bootstrap.servers': kafka_bootstrap_servers,
            'client.id': 'audit-logger'
        })

        self.logger = logging.getLogger('audit')
        self.logger.setLevel(logging.INFO)

        # File handler for local audit logs
        handler = logging.FileHandler('audit.log')
        formatter = logging.Formatter('%(asctime)s - %(message)s')
        handler.setFormatter(formatter)
        self.logger.addHandler(handler)

```

```

def log_access(self, user: str, resource: str, action: str,
               result: str, metadata: Dict = None):
    """
    Log access attempt

    Args:
        user: Username or service account
        resource: Resource being accessed
        action: Action performed (read, write, delete, etc.)
        result: Result (success, denied, error)
        metadata: Additional context
    """
    audit_record = {
        'event_type': 'access',
        'timestamp': datetime.now().isoformat(),
        'user': user,
        'resource': resource,
        'action': action,
        'result': result,
        'metadata': metadata or {},
        'checksum': None
    }

    # Add checksum for integrity
    audit_record['checksum'] = self._calculate_checksum(audit_record)

    # Log locally
    self.logger.info(json.dumps(audit_record))

    # Send to Kafka for centralized logging
    self.producer.produce(
        'audit-logs',
        key=user.encode('utf-8'),
        value=json.dumps(audit_record).encode('utf-8')
    )
    self.producer.poll(0)

def log_data_access(self, user: str, table: str, query: str,
                   row_count: int, duration_ms: float):
    """Log database query"""
    self.log_access(
        user=user,
        resource=f'table:{table}',
        action='query',
        result='success',
        metadata={
            'query': query[:500], # Truncate long queries
            'row_count': row_count,
            'duration_ms': duration_ms
        }
    )

def log_data_modification(self, user: str, table: str,

```

```

        operation: str, rows_affected: int):
    """Log data modification (INSERT, UPDATE, DELETE)"""
    self.log_access(
        user=user,
        resource=f'table:{table}',
        action=operation,
        result='success',
        metadata={
            'rows_affected': rows_affected
        }
    )

def log_authentication(self, user: str, method: str, result: str,
                        ip_address: str = None):
    """Log authentication attempt"""
    audit_record = {
        'event_type': 'authentication',
        'timestamp': datetime.now().isoformat(),
        'user': user,
        'method': method,
        'result': result,
        'ip_address': ip_address,
        'checksum': None
    }

    audit_record['checksum'] = self._calculate_checksum(audit_record)

    self.logger.info(json.dumps(audit_record))
    self.producer.produce('audit-logs',
value=json.dumps(audit_record).encode('utf-8'))
    self.producer.poll(0)

def log_permission_change(self, admin: str, user: str,
                          permission: str, action: str):
    """Log permission/role changes"""
    audit_record = {
        'event_type': 'permission_change',
        'timestamp': datetime.now().isoformat(),
        'admin': admin,
        'target_user': user,
        'permission': permission,
        'action': action, # 'grant' or 'revoke'
        'checksum': None
    }

    audit_record['checksum'] = self._calculate_checksum(audit_record)

    self.logger.info(json.dumps(audit_record))
    self.producer.produce('audit-logs',
value=json.dumps(audit_record).encode('utf-8'))
    self.producer.poll(0)

def _calculate_checksum(self, record: Dict) -> str:
    """Calculate checksum for audit record integrity"""

```

```

        # Remove checksum field if exists
        record_copy = record.copy()
        record_copy.pop('checksum', None)

        # Sort keys for consistent hashing
        record_str = json.dumps(record_copy, sort_keys=True)

        # SHA-256 hash
        return hashlib.sha256(record_str.encode('utf-8')).hexdigest()

def verify_integrity(self, record: Dict) -> bool:
    """Verify audit record hasn't been tampered with"""
    stored_checksum = record.get('checksum')
    if not stored_checksum:
        return False

    calculated_checksum = self._calculate_checksum(record)
    return stored_checksum == calculated_checksum

# Example usage
if __name__ == "__main__":
    audit = AuditLogger('localhost:9092')

    # Log various events
    audit.log_access(
        user='alice',
        resource='kafka://user-events',
        action='read',
        result='success'
    )

    audit.log_data_access(
        user='bob',
        table='orders',
        query='SELECT * FROM orders WHERE amount > 1000',
        row_count=542,
        duration_ms=125.3
    )

    audit.log_data_modification(
        user='system',
        table='users',
        operation='UPDATE',
        rows_affected=1
    )

    audit.log_authentication(
        user='charlie',
        method='SASL/SCRAM',
        result='success',
        ip_address='192.168.1.100'
    )

```

7. Data Masking & PII Protection {#data-masking}

Field-Level Encryption & Masking

File: `data_masking.py`

```
import re
import hashlib
from typing import Any, Callable
import random

class DataMasker:
    """
    Mask or encrypt sensitive data fields
    """

    @staticmethod
    def mask_email(email: str) -> str:
        """
        Mask email address
        Example: john.doe@example.com -> j***@example.com
        """
        if not email or '@' not in email:
            return email

        local, domain = email.split('@')
        if len(local) <= 1:
            masked_local = local
        else:
            masked_local = local[0] + '***'

        return f'{masked_local}@{domain}'

    @staticmethod
    def mask_phone(phone: str) -> str:
        """
        Mask phone number
        Example: +1-555-123-4567 -> +1-555-***-**67
        """
        if not phone:
            return phone

        # Extract digits only
        digits = re.sub(r'\D', '', phone)

        if len(digits) < 4:
            return '***'

        # Show last 2 digits
        masked = '*' * (len(digits) - 2) + digits[-2:]
```

```

# Try to preserve format
result = phone
for i, char in enumerate(phone):
    if char.isdigit():
        digit_index = len([c for c in phone[:i] if c.isdigit()])
        if digit_index < len(masked):
            result = result[:i] + masked[digit_index] + result[i+1:]

    return result

@staticmethod
def mask_ssn(ssn: str) -> str:
    """
    Mask Social Security Number
    Example: 123-45-6789 -> ***-**-6789
    """
    if not ssn:
        return ssn

    # Show last 4 digits only
    digits = re.sub(r'\D', '', ssn)
    if len(digits) < 4:
        return '***'

    return f'***-**-{{digits[-4:]}}'

@staticmethod
def mask_credit_card(cc: str) -> str:
    """
    Mask credit card number
    Example: 4111-1111-1111-1111 -> ****-****-****-1111
    """
    if not cc:
        return cc

    # Show last 4 digits
    digits = re.sub(r'\D', '', cc)
    if len(digits) < 4:
        return '****'

    masked_digits = '*' * (len(digits) - 4) + digits[-4:]

    # Preserve format
    result = cc
    digit_count = 0
    for i, char in enumerate(cc):
        if char.isdigit():
            if digit_count < len(masked_digits):
                result = result[:i] + masked_digits[digit_count] +
result[i+1:]
                digit_count += 1

    return result

```

```

@staticmethod
def pseudonymize(value: str, salt: str = 'default-salt') -> str:
    """
    Pseudonymize value using hash
    Same input always produces same output (deterministic)
    """
    if not value:
        return value

    hash_input = f'{value}:{salt}'
    hash_value = hashlib.sha256(hash_input.encode('utf-8')).hexdigest()

    # Return first 16 characters
    return hash_value[:16]

@staticmethod
def randomize_date(date_str: str, days_variance: int = 30) -> str:
    """
    Randomize date within variance
    Useful for birthdates, maintaining age group
    """
    from datetime import datetime, timedelta

    if not date_str:
        return date_str

    try:
        date_obj = datetime.fromisoformat(date_str)
        variance = random.randint(-days_variance, days_variance)
        randomized = date_obj + timedelta(days=variance)
        return randomized.isoformat()
    except:
        return date_str

@staticmethod
def apply_masking_rules(record: dict, rules: dict) -> dict:
    """
    Apply masking rules to record

    Args:
        record: Data record
        rules: Dict mapping field names to masking functions

    Example:
        rules = {
            'email': DataMasker.mask_email,
            'phone': DataMasker.mask_phone,
            'ssn': DataMasker.mask_ssn
        }
    """
    masked_record = record.copy()

    for field, masking_func in rules.items():
        if field in masked_record and masked_record[field]:

```

```

        masked_record[field] = masking_func(masked_record[field])

    return masked_record

class ConditionalMasker:
    """
    Apply masking based on user role/permissions
    """

    def __init__(self, user_role: str):
        self.user_role = user_role

        # Define masking rules per role
        self.role_rules = {
            'analyst': {
                'email': DataMasker.mask_email,
                'phone': DataMasker.mask_phone,
                'ssn': DataMasker.mask_ssn,
                'credit_card': DataMasker.mask_credit_card
            },
            'engineer': {
                'ssn': DataMasker.mask_ssn,
                'credit_card': DataMasker.mask_credit_card
            },
            'admin': {} # No masking for admin
        }

    def mask_record(self, record: dict) -> dict:
        """Apply role-appropriate masking"""
        rules = self.role_rules.get(self.user_role, {})
        return DataMasker.apply_masking_rules(record, rules)

# Example usage
if __name__ == "__main__":
    # Sample record
    record = {
        'user_id': '12345',
        'name': 'John Doe',
        'email': 'john.doe@example.com',
        'phone': '+1-555-123-4567',
        'ssn': '123-45-6789',
        'credit_card': '4111-1111-1111-1111'
    }

    # Define masking rules
    masking_rules = {
        'email': DataMasker.mask_email,
        'phone': DataMasker.mask_phone,
        'ssn': DataMasker.mask_ssn,
        'credit_card': DataMasker.mask_credit_card
    }

    # Apply masking
    masked = DataMasker.apply_masking_rules(record, masking_rules)

```

```

print("Original:", record)
print("Masked:", masked)

# Conditional masking by role
analyst_masker = ConditionalMasker('analyst')
analyst_view = analyst_masker.mask_record(record)

admin_masker = ConditionalMasker('admin')
admin_view = admin_masker.mask_record(record)

print("\nAnalyst view:", analyst_view)
print("Admin view:", admin_view)

```

Dynamic Data Masking in Queries

PostgreSQL Row-Level Security:

```

-- Enable RLS
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Policy: Users can only see their own data
CREATE POLICY user_isolation ON users
    FOR SELECT
    USING (user_id = current_user);

-- Policy: Admins can see all data
CREATE POLICY admin_all_access ON users
    FOR ALL
    TO admin_role
    USING (true);

-- Policy: Analysts see masked data
CREATE POLICY analyst_masked_view ON users
    FOR SELECT
    TO analyst_role
    USING (true)
    WITH CHECK (
        -- Custom masking function
        mask_sensitive_fields(users.*)
    );

-- Masking function
CREATE OR REPLACE FUNCTION mask_email(email TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN SUBSTRING(email FROM 1 FOR 1) || '***@' ||
        SUBSTRING(email FROM POSITION('@' IN email) + 1);
END;
$$ LANGUAGE plpgsql;

```

8. Security Best Practices {#best-practices}

Security Checklist

- ☒ Network Security:
 - ☐ Use VPC/private networks
 - ☐ Implement network segmentation
 - ☐ Configure security groups/firewalls
 - ☐ Enable VPN for remote access
 - ☐ Use bastion hosts for SSH access
- ☒ Authentication:
 - ☐ Enable multi-factor authentication (MFA)
 - ☐ Use strong password policies
 - ☐ Implement SSO where possible
 - ☐ Rotate credentials regularly
 - ☐ Use service accounts for applications
- ☒ Authorization:
 - ☐ Implement least privilege principle
 - ☐ Use RBAC
 - ☐ Regular access reviews
 - ☐ Separate production/development access
- ☒ Encryption:
 - ☐ Encrypt data at rest
 - ☐ Encrypt data in transit (TLS 1.2+)
 - ☐ Use strong encryption algorithms (AES-256)
 - ☐ Manage keys securely (KMS/HSM)
 - ☐ Rotate encryption keys
- ☒ Audit & Monitoring:
 - ☐ Enable comprehensive audit logging
 - ☐ Monitor for suspicious activity
 - ☐ Set up alerts for security events
 - ☐ Regularly review audit logs
 - ☐ Implement SIEM integration
- ☒ Data Protection:
 - ☐ Classify data by sensitivity
 - ☐ Mask/encrypt PII
 - ☐ Implement data retention policies
 - ☐ Secure data deletion processes
 - ☐ Regular backups
- ☒ Application Security:
 - ☐ Input validation
 - ☐ SQL injection prevention
 - ☐ XSS protection
 - ☐ CSRF protection
 - ☐ Secure dependencies (vulnerability scanning)

- ☒ Compliance:
 - ☐ Document compliance requirements
 - ☐ Implement required controls
 - ☐ Regular compliance audits
 - ☐ Staff training
 - ☐ Incident response plan

Incident Response Plan

File: **incident_response.md**

```
# Data Security Incident Response Plan

## 1. Detection & Analysis

### Indicators of Compromise:
- Unusual access patterns
- Large data exports
- Failed authentication attempts
- Unauthorized permission changes
- Anomalous network traffic

### Initial Assessment:
1. Confirm incident is genuine
2. Determine scope and severity
3. Document timeline
4. Identify affected systems/data

## 2. Containment

### Short-term:
- Isolate affected systems
- Revoke compromised credentials
- Block suspicious IP addresses
- Disable compromised accounts

### Long-term:
- Patch vulnerabilities
- Update security controls
- Review and update access policies

## 3. Eradication

- Remove malware/backdoors
- Close security gaps
- Reset all potentially compromised credentials
- Update firewall rules

## 4. Recovery
```

- Restore from clean backups
- Verify system integrity
- Monitor for recurrence
- Gradual service restoration

5. Post-Incident

- Conduct root cause analysis
- Document lessons learned
- Update incident response procedures
- Staff training
- Notify stakeholders/regulators if required

Security Hardening Configuration

Kafka Hardening:

```
# Disable auto-create topics
auto.create.topics.enable=false

# Require authentication
security.inter.broker.protocol=SASL_SSL

# Disable unnecessary protocols
listeners=SASL_SSL://0.0.0.0:9093

# Enable authorization
authorizer.class.name=kafka.security.authorizer.AclAuthorizer

# Restrict broker access
super.users=User:admin

# Enable audit logging
kafka.authorizer.logger.name=kafka.authorizer.logger

# Rate limiting
quota.producer.default=10485760
quota.consumer.default=10485760

# Connection limits
max.connections.per.ip=100
```

Conclusion

Security and governance are critical for data ingestion pipelines:

1. **Encrypt everything:** Data at rest and in transit
2. **Implement strong authentication & authorization**
3. **Track data lineage** for transparency

4. **Enforce data quality** standards
 5. **Comply with regulations** (GDPR, CCPA, HIPAA)
 6. **Comprehensive audit logging** for accountability
 7. **Protect PII** through masking and encryption
 8. **Regular security audits** and updates
-

Version: 1.0